

Notes:

- HW1 graded, solutions posted
- HW2 due Oct 16
- No class on Tuesday
- Project description posted, proposals due Oct 21
- HW3 goes out today

Last class:

- `assert` , `try - except`
- unit testing
- `range` statements
- `pytest`
- parametrizing tests
- `xfails`
- `fixtures`

What are those "@" signs doing there?

Functions that take only data as their inputs and return data as their outputs are known as "first-order functions".

Functions that take functions as their input and return functions as their outputs are known as "higher-order functions".

In Python, we use "decorators" as functions that take other functions as inputs, but don't modify the inputs to those other functions.

```
In [3]: def twice(f):
        def result(x):
            return f(f(x))
        return result

    def f(i):
        return 2*i+3

    g1 = twice(f)

    g1(3)
```

Out[3]: 21

Here, `f` is a regular ("first-order") function, `twice` is a higher-order function, since it acts on a function as input and returns a function as output, and `g1` is the output of `twice` acting on `f`

Can now write:

```
In [4]: @twice
        def g2(i):
            return 2*i+3

    g2(3)
```

Out[4]: 21

Gives the same result as `g1` , but it's shorter code. Use `@twice` to say that we want to apply higher-order function given by `twice` . The resulting lower-order function will be called `g2` , and `twice` should act on the function given by `2*i+3`

In Python, we call a function like `twice` a **decorator**, and use "syntactic sugar" to denote their use with `@`

So `@pytest.mark.parametrize` and `@pytest.fixture` denote that we have higher-order functions in `pytest` that act on our tests, whose behaviour is determined by the additional inputs (if any) that we provide.

How do we effectively design tests?

Aim for coverage of all possible cases of input data.

In our "mean" example from last class, we wrote tests for different types of inputs and outputs. For some functions, it's important to also consider special cases where non-standard values are returned.

Consider `mymath.py` . This defines a function that returns the value of $\frac{\sin(x)}{x} \times \frac{\sin(y)}{y}$, substituting the correct limit that $\frac{\sin(x)}{x} \rightarrow 1$ as $x \rightarrow 0$

The python syntax here is:

```
if statement1:
    action1
elif statement2:
    action2
else:
    action3
```

where `action1` is executed if `statement1` is true, `action2` is executed if `statement1` is false, but `statement2` is true, and `action3` is executed if both `statement1` and `statement2` are false

Can have any number of `elif` lines, that are checked in sequence.

Our question: How do we define suitable tests for `sinc2d` ?

Aim to test each outcome of each if statement. That means we should test with `(x,y) = (0,0)` , with `x=0` but `y` not zero, with `y=0` , but `x` not zero, and with both `x` and `y` not zero

Use the fact that we know that $\sin(\frac{\pi}{2}) = 1$ and $\sin(\frac{3\pi}{2}) = -1$

To write these tests, use `@pytest.mark.parametrize` . See in `test_mymath.py`

```
[math-scott02:CMSC6950/Fall2025/Notes] smaclachlan% pytest test_mymath.py
===== test session starts =====
platform darwin -- Python 3.13.3, pytest-8.3.5, pluggy-1.6.0
rootdir: /Users/smaclachlan/Work/teaching/CMSC6950/Fall2025/Notes
plugins: mpi-pytest-2025.5.0, cov-7.0.0
collected 4 items

test_mymath.py .... [100%]

===== 4 passed in 0.02s =====
```

We sometimes refer to this style of testing as checking corner and edge cases.

The general case code here (often call the "interior case") is when $x \neq 0$ and $y \neq 0$. Important to test this, but also important to test all special cases.

Here, two special cases when $x = 0$ and $y = 0$. Call these "edge cases". When both are satisfied , we call this a "corner case".

Effective testing must test both edge and corner cases, in addition to interior cases

Unit testing like this is the first of 3 important class of testing:

1. Unit testing
2. Integration testing
3. Regression testing

Integration tests are tests of the functionality of composing multiple functions.

Back to `mymath.py` look at `addone` , `double` , `addone_then_double`

Can write some unit tests for `addone` and `double` . Testing `addone_then_double` is integration testing, since we test how two functions work together. See this in `test_mymath2.py`

```
===== test session starts =====
platform darwin -- Python 3.13.3, pytest-8.3.5, pluggy-1.6.0
rootdir: /Users/smaclachlan/Work/teaching/CMSC6950/Fall2025/Notes
plugins: mpi-pytest-2025.5.0, cov-7.0.0
collected 12 items

test_mymath2.py ..... [100%]

===== 12 passed in 0.02s =====
```

The distinction between "unit testing" and "integration testing" is not always clear, and is usually not important. We think of integration testing as testing how 2 units that we've written (which themselves may call other units of code that we've written or code from other libraries) interact. But, what's important is that we test all of the code that we wrote, and not what we call the tests

Regression testing is qualitatively different. For regression testing, we run larger segments of code and record the results. Then check that these values don't change when we run them again.

Do this for `sinc2d` . First compute some values:

```
In [5]: import mymath
        mymath.sinc2d(1,1)

Out[5]: 0.7080734182735712

In [6]: mymath.sinc2d(5,0.25)

Out[6]: -0.18979332974154567

In [7]: mymath.sinc2d(0, 17)

Out[7]: -0.05655279363997393
```

Add these values to my tests. Now need add tolerances, since they aren't simple values to express, and because different hardware might compute slightly different values.

```
[math-scott02:CMSC6950/Fall2025/Notes] smaclachlan% pytest -v test_mymath.py
===== test session starts =====
platform darwin -- Python 3.13.3, pytest-8.3.5, pluggy-1.6.0 -- /Users/smaclachlan/firedrake-June2025/bin/python3.13
cachedir: .pytest_cache
rootdir: /Users/smaclachlan/Work/teaching/CMSC6950/Fall2025/Notes
plugins: mpi-pytest-2025.5.0, cov-7.0.0
collected 7 items

test_mymath.py::test_sinc2d[0-0-1] PASSED [ 14%]
test_mymath.py::test_sinc2d[0-1.5707963267948966-0.6366197723675814] PASSED [ 28%]
test_mymath.py::test_sinc2d[4.71238898038469-0-0.2122065907891938] PASSED [ 42%]
test_mymath.py::test_sinc2d[4.71238898038469-1.5707963267948966-0.13509491152311703] PASSED [ 57%]
test_mymath.py::test_sinc2d_regression[1-1-0.7080734182735712] PASSED [ 71%]
test_mymath.py::test_sinc2d_regression[5-0.25-0.18979332974154567] PASSED [ 85%]
test_mymath.py::test_sinc2d_regression[0-17-0.05655279363997393] PASSED [100%]

===== 7 passed in 0.02s =====
```

We can install (using `pip install pytest-cov`) and use `pytest-cov` package to examine the coverage of our tests.

```
[math-scott02:CMSC6950/Fall2025/Notes] smaclachlan% pytest --cov=mymath test_mymath.py
===== test session starts =====
platform darwin -- Python 3.13.3, pytest-8.3.5, pluggy-1.6.0
rootdir: /Users/smaclachlan/Work/teaching/CMSC6950/Fall2025/Notes
plugins: mpi-pytest-2025.5.0, cov-7.0.0
collected 7 items

test_mymath.py ..... [100%]

===== tests coverage =====
_____ coverage: platform darwin, python 3.13.3-final-0 _____

Name      Stmts   Miss  Cover
-----
mymath.py    15      3    80%
TOTAL        15      3    80%

===== 7 passed in 0.04s =====
```

Tells us `test_mymath.py` covers 80% of the code in `mymath.py` . Which 80%?

Add `--cov-report term-missing` before tests:

```
[math-scott02:CMSC6950/Fall2025/Notes] smaclachlan% pytest --cov=mymath --cov-report term-missing test_mymath.py
===== test session starts =====
platform darwin -- Python 3.13.3, pytest-8.3.5, pluggy-1.6.0
rootdir: /Users/smaclachlan/Work/teaching/CMSC6950/Fall2025/Notes
plugins: mpi-pytest-2025.5.0, cov-7.0.0
collected 7 items

test_mymath.py ..... [100%]

===== tests coverage =====
_____ coverage: platform darwin, python 3.13.3-final-0 _____

Name      Stmts   Miss  Cover   Missing
-----
mymath.py    15      3    80%    16, 20, 24
TOTAL        15      3    80%

===== 7 passed in 0.04s =====
```

Tells us we're missing testing code on lines 16, 20, and 24 of `mymath.py` . Add more tests to command line:

Add `test_mymath2.py` to get:

```
[math-scott02:CMSC6950/Fall2025/Notes] smaclachlan% pytest --cov=mymath --cov-report term-missing test_mymath.py test_mymath2.py
===== test session starts =====
platform darwin -- Python 3.13.3, pytest-8.3.5, pluggy-1.6.0
rootdir: /Users/smaclachlan/Work/teaching/CMSC6950/Fall2025/Notes
plugins: mpi-pytest-2025.5.0, cov-7.0.0
collected 19 items

test_mymath.py ..... [ 36%]
test_mymath2.py ..... [100%]

===== tests coverage =====
_____ coverage: platform darwin, python 3.13.3-final-0 _____

Name      Stmts   Miss  Cover   Missing
-----
mymath.py    15      0   100%
-----
TOTAL        15      0   100%

===== 19 passed in 0.05s =====
```

Now see 100% coverage

Some projects use "test driven development", where we *first* write tests, then write functions of code, but this is just a question of style

Many scientific software packages *do* make use of a continuous integration / continuous delivery (CI/CD) approach to releasing software. Understand this is in contrast to "old-school" software releases:

Windows 3.0 was released in May 1990

Windows 3.1 was released in April 1992

Windows NT3.1 was released in July 1993

Windows 3.11 was released in November 1993

We used to have big discrete releases on infrequent schedule.

In Continuous Integration, we develop code on branches, then merge branches into main on a regular (frequent) basis, when they are "ready".

A key technique in this approach is that of automated testing. Before any code can be merged into main, it must

1. Pass all tests (including new tests for the new code)
2. Be reviewed for coding standards, correctness, etc.